

Advanced Statistical Learning

Model Inference & Averaging

Xiaolei Lin

School of Data Science
Fudan University

Dec 1, 2025

Outline

Maximum Likelihood Methods & Bootstrap

Bayesian Methods

The EM Algorithm

MCMC - Sampling from Posterior

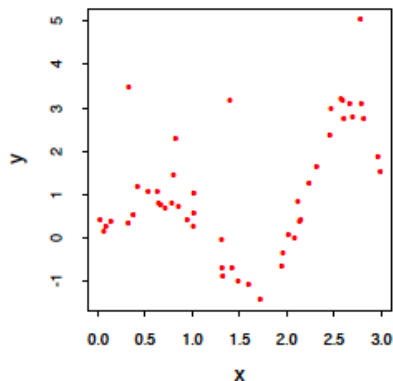
Bagging

Model Averaging & Stacking

Stochastic Search: Bumping

Maximum Likelihood Methods & Bootstrap

Example - Data



Training data

$$Z = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

- ▶ X - 1-dim input
- ▶ Y - continuous output
- ▶ $N = 50$

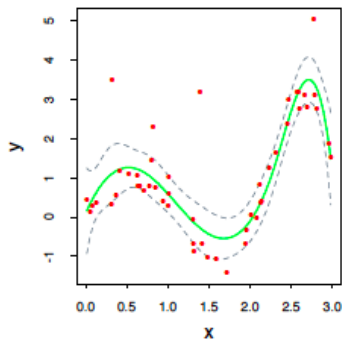
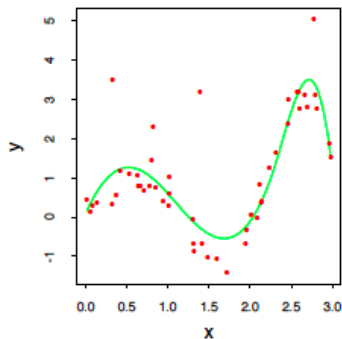
Example - Model

Consider a cubic spline with 3 knots located at the quartiles of the X values

$$Y = \sum_{j=1}^J \beta_j h_j(X) + \epsilon$$

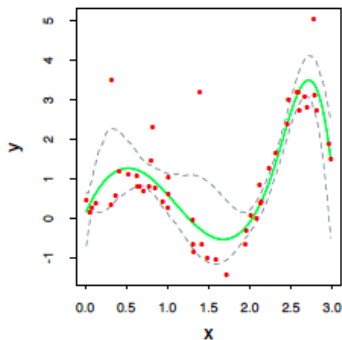
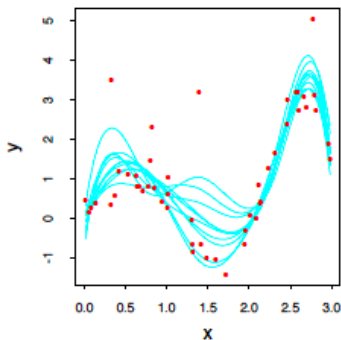
Let $u(x) = \sum_{j=1}^J \beta_j h_j(X)$

Example - Maximum Likelihood Methods



- ▶ Left: $\hat{\mu}(x)$
- ▶ Right: pointwise 95% confidence band $\hat{\mu}(x) \pm 1.96\hat{s}e(\hat{\mu}(x))$

Example - Bootstrap



- ▶ Left: $\hat{\mu}^{*1}(x), \hat{\mu}^{*2}(x), \dots, \hat{\mu}^{*B}(x)$
- ▶ Right: 2.5% & 97.5% percentile of $\hat{\mu}^{*1}(x), \hat{\mu}^{*2}(x), \dots, \hat{\mu}^{*B}(x)$

Maximum Likelihood Inference

Specify a probability density / mass function for observations

$$z_i \sim g_{\theta}(z)$$

Likelihood function (function of θ given Z)

$$L(\theta; Z) = \prod_{i=1}^N g_{\theta}(z_i)$$

Easy to deal with the logarithm of $L(\theta; Z)$

$$l(\theta; Z) = \sum_{i=1}^N l(\theta; z_i) = \sum_{i=1}^N \log g_{\theta}(z_i)$$

Maximum Likelihood estimate

$$\hat{\theta} = \operatorname{argmax}_{\theta} l(\theta; Z)$$

Maximum Likelihood Inference

Score function

$$\dot{l}(\theta; Z) = \sum_{i=1}^N \dot{l}(\theta; z_i), \text{ where } \dot{l}(\theta; z_i) = \frac{\partial l(\theta; z_i)}{\partial \theta}$$

Assuming likelihood takes its maximum in the interior of the parameter space, $\dot{l}(\hat{\theta}; Z) = 0$

Information matrix

$$I(\theta) = - \sum_{i=1}^N \frac{\partial^2 l(\theta; z_i)}{\partial \theta \partial \theta^T}$$

- ▶ Observed information $I(\theta) |_{\theta=\hat{\theta}}$
- ▶ Fisher information $i(\theta) = E_{\theta}[I(\theta)]$

Maximum Likelihood Inference

Sampling distribution of MLE $\hat{\theta}$

$$\hat{\theta} \rightarrow \mathcal{N}(\theta_0, i(\theta_0)^{-1}) \text{ as } N \rightarrow \infty$$

where θ_0 denotes the true value of θ

Can be approximated by

$$\mathcal{N}(\hat{\theta}, i(\hat{\theta})^{-1}) \text{ or } \mathcal{N}(\hat{\theta}, I(\hat{\theta})^{-1})$$

Estimates for standard error of $\hat{\theta}_j$

$$\sqrt{i(\hat{\theta})_{jj}^{-1}} \text{ or } \sqrt{I(\hat{\theta})_{jj}^{-1}}$$

Maximum Likelihood Inference

Confidence interval

$$\hat{\theta}_j - z^{1-\alpha} \cdot \sqrt{i(\hat{\theta})_{jj}^{-1}} \text{ or } \hat{\theta}_j - z^{1-\alpha} \cdot \sqrt{l(\hat{\theta})_{jj}^{-1}}$$

Joint confidence region

$$\{\theta; 2(l(\hat{\theta}) - l(\theta)) \leq \chi_p^{2, (1-2\alpha)}\}$$

Since $2[l(\hat{\theta}) - l(\theta_0)] \sim \chi_p^2$

Bayesian Methods

Bayesian Approach

Posterior distribution of θ given prior and data

$$p(\theta \mid Z) = \frac{p(Z|\theta)p(\theta)}{\int p(Z|\theta')p(\theta')d\theta'}$$

Predictive distribution for new data

$$p(z^{new} \mid Z) = \int p(z^{new} \mid \theta)p(\theta \mid Z)d\theta$$

However, by Maximum Likelihood methods, the predictive distribution for new data z^{new} is $p(z^{new} \mid \hat{\theta}_{MLE})$, which does not account for the uncertainty in estimating θ

Back to Example

$$Y = \sum_{j=1}^J \beta_j h_j(X) + \epsilon, \text{ with } \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Let $u(x) = \sum_{j=1}^J \beta_j h_j(X)$, B be the design matrix with ij_{th} element $h_j(x_i)$

Prior on $\beta = (\beta_1, \dots, \beta_p)^\top$: $\beta \sim \mathcal{N}(0, \tau \Sigma)$

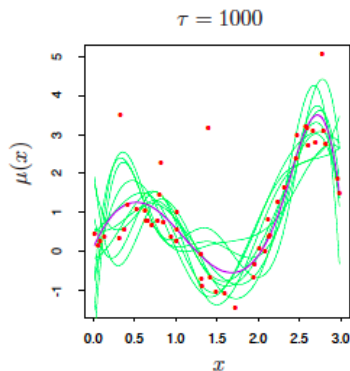
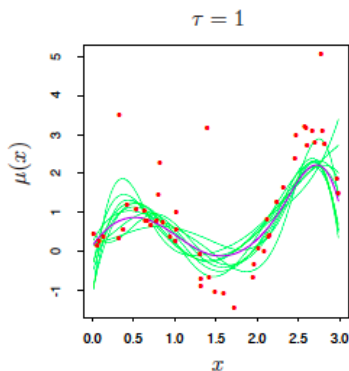
Data likelihood: $p(y \mid X, \beta) = \mathcal{N}(y; B\beta, \sigma^2 I_n)$

Posterior distribution for β :

$$p(\beta \mid Z) = p(\beta \mid X, y) = \frac{p(y \mid X, \beta) p(\beta)}{p(y \mid X)} = \mathcal{N}(\beta; A^{-1} B^\top y, A^{-1} \sigma^2)$$

with $A = B^\top B + \frac{\sigma^2}{\tau} \Sigma^{-1}$

Distribution of Predicted Curves

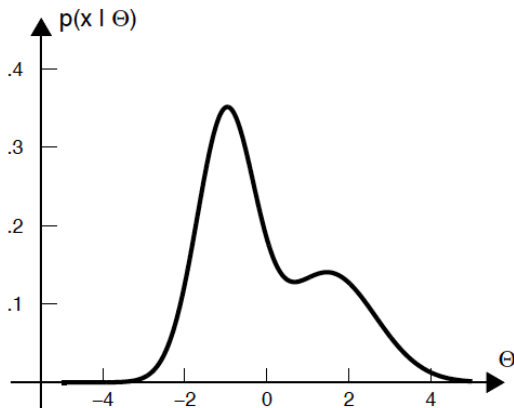


- ▶ $\tau \rightarrow \infty$ - noninformative prior
- ▶ For Gaussian models, bootstrap agree with non-informative Bayes
- ▶ How about σ ?

The EM Algorithm

Limitations of Normal Distribution

Gaussian distributions are not always impressive



Gaussian Mixture Models (GMM)

Mathematical definition

$$p(x \mid \theta) = \sum_{k=1}^K \pi_k \cdot N(x_k; \mu_k, \Sigma_k)$$

where $\sum_{k=1}^K \pi_k = 1$ and $\pi_k \geq 0$ for $k = 1, \dots, K$

and $\theta = (\mu_1, \dots, \mu_K, \Sigma_1, \dots, \Sigma_K, \pi_1, \dots, \pi_K)$

Parameter Estimation for GMM

Given n independent training samples $x_1, x_2, \dots, x_n$ from a GMM, can still use MLE to estimate θ , but...

Assume K is fixed and known, log likelihood of the data is

$$l(\theta; x) = \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k N(x_i; \mu_k, \Sigma_k) \right)$$

Let's try to maximize it

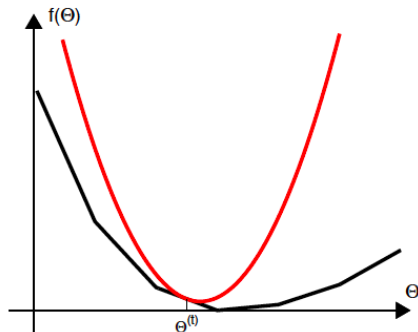
- ▶ By eyeballing
- ▶ Analytically
- ▶ Numerically

The MM Algorithm - Minimization

To minimize an objective function $f(\theta)$

- ▶ The MM algorithm is a prescription for constructing optimization algorithms
- ▶ An MM algorithm creates a surrogate function that majorizes the objective function.
- ▶ When the surrogate function is minimized, the objective function is decreased
- ▶ For minimization, $\text{MM} \equiv \text{majorize}$

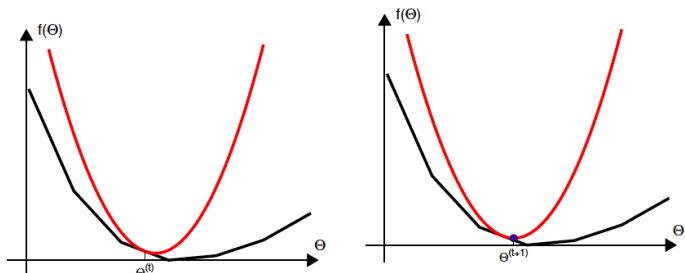
Majorization



A function $g(\theta; \theta^{(t)})$ majorizes the function $f(\theta)$ at $\theta^{(t)}$ if

- ▶ $f(\theta^{(t)}) = g(\theta^{(t)}; \theta^{(t)})$
- ▶ $f(\theta) \leq g(\theta; \theta^{(t)})$ for all θ

Majorization



If $f(\theta)$ is difficult to minimize, find $g(\theta; \theta^{(t)})$, which is easy to minimize. Let

$$\theta^{(t+1)} = \operatorname{argmin}_{\theta} g(\theta; \theta^{(t)})$$

Majorization - Descent Property

MM minimization algorithm satisfied the descent property as

$$\begin{aligned}f(\theta^{(t+1)}) &\leq g(\theta^{(t+1)}; \theta^{(t)}) \\&\leq g(\theta^{(t)}; \theta^{(t)}) \\&= f(\theta^{(t)})\end{aligned}$$

In summary

$$f(\theta^{(t+1)}) \leq f(\theta^{(t)})$$

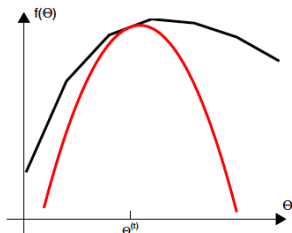
The descent property makes the MM algorithm very stable - algorithms converges to local minima or saddle point

The MM Algorithm - Maximization

To maximize an objective function $f(\theta)$

- ▶ An MM algorithm creates a surrogate function that minorizes the objective function.
- ▶ When the surrogate function is maximized, the objective function is increased
- ▶ For maximization, $\text{MM} \equiv \text{minorize}$

Minorization



Red curve minorize the black curve

A function $g(\theta; \theta^{(t)})$ minorizes the function $f(\theta)$ at $\theta^{(t)}$ if

- ▶ $f(\theta^{(t)}) = g(\theta^{(t)}; \theta^{(t)})$
- ▶ $f(\theta) \geq g(\theta; \theta^{(t)})$ for all θ

Big Question

How do we majorize / minorize a function?

Some generic tricks and tools

- ▶ Jensen's inequality
- ▶ Chord above the graph property of a convex function
- ▶ Supporting hyperplane property of a convex function
- ▶ Quadratic upper bound principle
- ▶ Arithmetic-geometric mean inequality
- ▶ Cauchy-Schwartz inequality

We'll only focus on Jensen's inequality here

Jensen's Inequality

You've probably minorized via Jensen's Inequality without noticing it

Remember Jensen's Inequality

- ▶ $h(\cdot)$ be a concave function
- ▶ $\pi_1, \dots, \pi_K$ be K non-negative numbers, with $\sum_{k=1}^K \pi_k = 1$
- ▶ K arbitrary numbers $a_1, \dots, a_K$

Then

$$h\left(\sum_{k=1}^K \pi_k a_k\right) \geq \sum_{k=1}^K \pi_k h(a_k)$$

Expectation - Maximization

- ▶ The EM algorithm is a MM algorithm
- ▶ EM uses Jensen's inequality to minorize the log likelihood

Step 1

$$\begin{aligned}l(\theta; X) &= \log \left(\sum_{j=1}^{n_z} p(X, Z = z_j \mid \theta) \right) \\&= \log \left(\sum_{j=1}^{n_z} f^{(t)}(Z = z_j) \frac{p(X, Z = z_j \mid \theta)}{f^{(t)}(Z = z_j)} \right) \\&\geq \sum_{j=1}^{n_z} f^{(t)}(Z = z_j) \log \left(\frac{p(X, Z = z_j \mid \theta)}{f^{(t)}(Z = z_j)} \right)\end{aligned}$$

Summary of step 1, we've got

$$l(\theta; Z) \geq \sum_{j=1}^{n_z} f^{(t)}(Z = z_j) \log \left(\frac{p(X, Z=z_j|\theta)}{f^{(t)}(Z=z_j)} \right)$$

Expectation - Maximization

Step 2: find $f^{(t)}(Z)$

The lower bound must touch the log likelihood at $\theta^{(t)}$

$$l(\theta^{(t)}; X) = \sum_{j=1}^{n_z} f^{(t)}(Z = z_j) \log \left(\frac{p(X, Z=z_j | \theta^{(t)})}{f^{(t)}(Z=z_j)} \right)$$

From this we can calculate $f^{(t)}(Z)$

$$f^{(t)}(Z) = p(Z | X, \theta^{(t)})$$

The log likelihood is minorized by

$$g(\theta; \theta^{(t)}) = \sum_{j=1}^{n_z} p(Z = z_j | X, \theta^{(t)}) \log \left(\frac{p(X, Z=z_j | \theta)}{p(Z=z_j | X, \theta^{(t)})} \right)$$

Expectation - Maximization

Step 3: find & max $g(\theta; \theta^{(t)})$

What are the Z 's and where did they come from?

- ▶ Z is a random variable whose pdf conditioned on X is completely determined by θ
- ▶ Choice of Z should make the maximization step easy

Two Component GMM

$$Y_1 \sim N(\mu_1, \sigma_1^2)$$

$$Y_2 \sim N(\mu_2, \sigma_2^2)$$

$$Y = (1 - \Delta) \cdot Y_1 + \Delta \cdot Y_2$$

where $\Delta = \{0, 1\}$ with $Pr(\Delta = 1) = \pi$

Equivalent two-step representation

- ▶ Generate a Δ with probability π
- ▶ Depending on the outcome of Δ , deliver either Y_1 or Y_2

Two Component GMM

Let $\phi_\theta(x)$ denote the normal density with parameter $\theta = (\mu, \sigma^2)$, then the density of Y is

$$P_Y(y) = (1 - \pi)\phi_{\theta_1}(y) + \pi\phi_{\theta_2}(y)$$

where $\theta = (\pi, \theta_1, \theta_2) = (\pi, \mu_1, \sigma_1^2, \mu_2, \sigma_2^2)$

The log likelihood of the data

$$l(\theta; Y) = \sum_{i=1}^N \log[(1 - \pi)\phi_{\theta_1}(y_i) + \pi\phi_{\theta_2}(y_i)]$$

Two Component GMM

Introduce latent variables Δ_i taking values 0 or 1

- ▶ If $\Delta_i = 1$, Y_i comes from model 2
- ▶ Otherwise it comes from model 1

Suppose we know the values of Δ_i , the log likelihood can be written as

$$l(\theta; Y, \Delta) = \sum_{i=1}^N [(1 - \Delta_i) \log \phi_{\theta_1}(y_i) + \Delta_i \log \phi_{\theta_2}(y_i)] + \sum_{i=1}^N [(1 - \Delta_i) \log(1 - \pi) + \Delta_i \log \pi]$$

- ▶ The MLE of μ_1 and σ_1^2 would be the sample mean and variance for those data with $\Delta_i = 0$
- ▶ The MLE of μ_2 and σ_2^2 would be the sample mean and variance for those data with $\Delta_i = 1$
- ▶ The MLE of π would be the proportion of $\Delta_i = 1$

Two Component GMM

However, Δ_i are unknown, and we substitute with its expected value

$$\gamma_i(\theta) = E(\Delta_i \mid \theta, Y) = Pr(\Delta_i = 1 \mid \theta, Y)$$

- ▶ γ_i is called responsibility of model 2 for observation i
- ▶ posterior of the probability that observation i belongs to model 2 given the data and current estimate

Two Component GMM

For each iteration....

Expectation step

- ▶ soft assignment of each observation to each model
- ▶ the current parameter estimates are used to assign responsibilities according to the relative density of the training points under each model

Maximization step

- ▶ these responsibilities are used in weighted maximum-likelihood fits to update the parameter estimates

Initial values for $\mu_1, \sigma_1^2, \mu_2, \sigma_2^2, \pi$

EM Algorithm for Two-component GMM

1. Take initial guesses for the parameters $(\mu_1, \sigma_1^2, \mu_2, \sigma_2^2, \pi)$
2. Expectation Step: compute the responsibilities

$$\hat{\gamma}_i = \frac{\hat{\pi} \phi_{\hat{\theta}_2}(y_i)}{(1-\hat{\pi})\phi_{\hat{\theta}_1}(y_i) + \hat{\pi}\phi_{\hat{\theta}_2}(y_i)}$$

3. Maximization Step: compute the weighted means and variances and the mixing probability

$$\hat{\mu}_1 = \frac{\sum_i (1-\hat{\gamma}_i)y_i}{\sum_i (1-\hat{\gamma}_i)}, \quad \hat{\sigma}_1^2 = \frac{\sum_i (1-\hat{\gamma}_i)(y_i - \hat{\mu}_1)^2}{\sum_i (1-\hat{\gamma}_i)}$$

$$\hat{\mu}_2 = \frac{\sum_i \hat{\gamma}_i y_i}{\sum_i \hat{\gamma}_i}, \quad \hat{\sigma}_2^2 = \frac{\sum_i \hat{\gamma}_i (y_i - \hat{\mu}_2)^2}{\sum_i \hat{\gamma}_i}$$

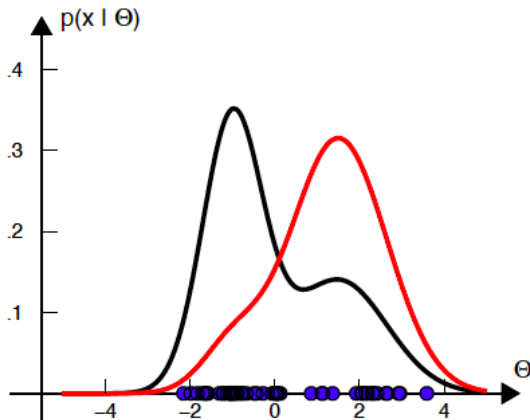
$$\hat{\pi} = \sum_i \frac{\hat{\gamma}_i}{N}$$

4. Iterate steps 2 and 3 until convergence

EM Visualization

A toy example to visualize EM

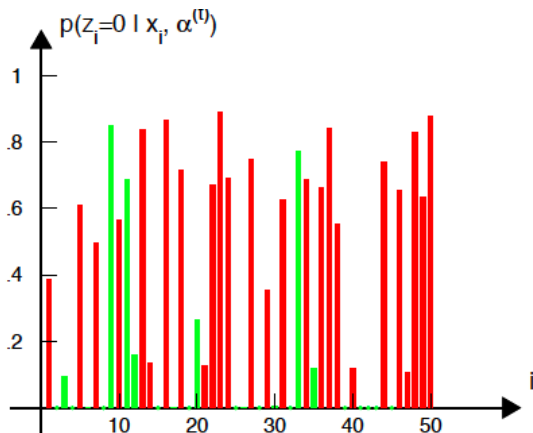
Let α denote the probability that Y_i comes from component 1



Initial guess of distribution
with $\alpha^{(0)} = .1$

EM Visualization

Graph shows the probability that each observation comes from component 1, given $a^{(0)}$

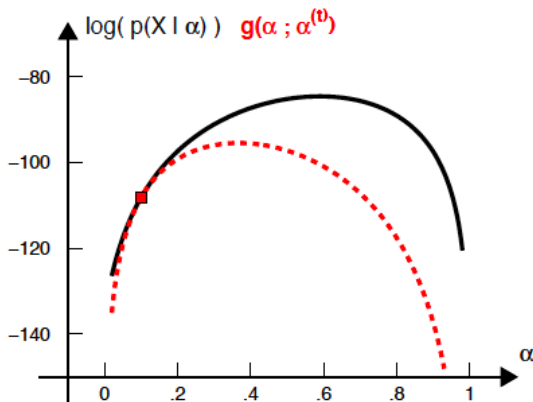


► Red: sample really generated by component 1

► Green: sample really generated by component 2

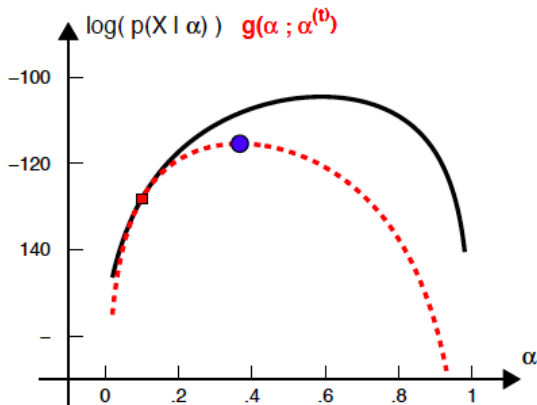
EM Visualization

Compute the expectation minorizing the log likelihood at $a^{(0)} = 0.1$



EM Visualization

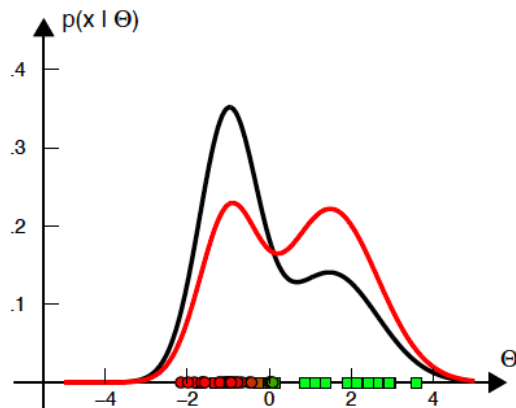
Calculate maximum of $g(a; a^{(0)})$



Maximum of $g(a; a^{(0)})$ gives $a^{(1)} = 0.3672$

EM Visualization

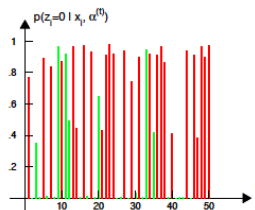
End of iteration 1



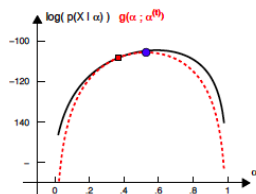
The estimate of the GMM with $a^{(1)} = 0.3672$

EM Visualization

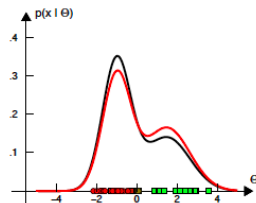
Iteration 2



Posterior probabilities



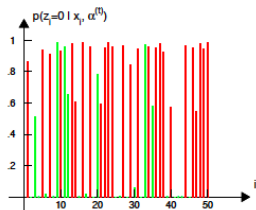
$$g(\alpha; \alpha^{(1)})$$
$$\alpha^{(2)} = .5287$$



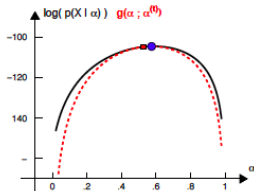
Current GMM estimate

EM Visualization

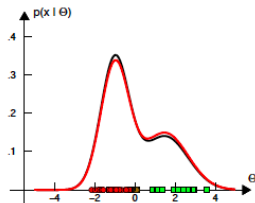
Iteration 3



Posterior probabilities

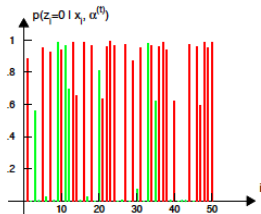


$$g(\alpha; \alpha^{(2)})$$
$$\alpha^{(3)} = .5748$$

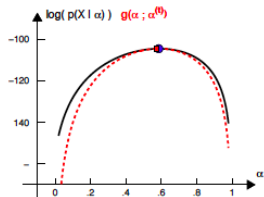


Current GMM estimate

EM Visualization

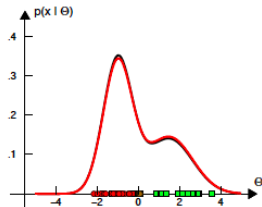


Posterior probabilities



$$g(\alpha; \alpha^{(3)})$$

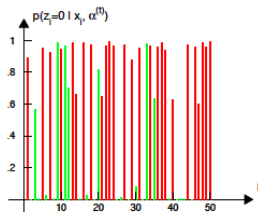
$$\alpha^{(4)} = .5859$$



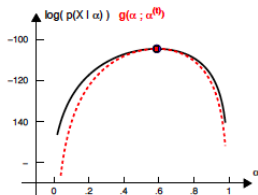
Current GMM estimate

EM Visualization

Iteration 5

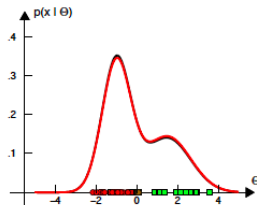


Posterior probabilities



$$g(\alpha; \alpha^{(4)})$$

$$\alpha^{(5)} = .5885$$



Current GMM estimate

MCMC

- Sampling from Posterior

Markov Chain Monte Carlo Method

Aim

- ▶ Generate independent samples $\{x^{(r)}\}_{r=1}^R$ from a pdf $p(x)$
- ▶ Can then use $x^{(r)}$ to estimate expectations of functions under this distribution

$$E[\phi(x)] = \int_x \phi(x)p(x)dx \approx \frac{1}{R} \sum_{r=1} \phi(x^{(r)})$$

Not an easy task

- ▶ Sampling from $p(x)$ is hard in general
- ▶ Especially when $x \in \mathbb{R}^p$ and p is large

Common approach

- ▶ Monte Carlo Markov Chain methods such as Metropolis-Hastings and Gibbs sampling

MCMC Assumptions

- ▶ Want to draw samples from $p(x)$
- ▶ Can evaluate $p(x)$ within a normalization factor
- ▶ That is, can evaluate a function $p^*(x)$ such that

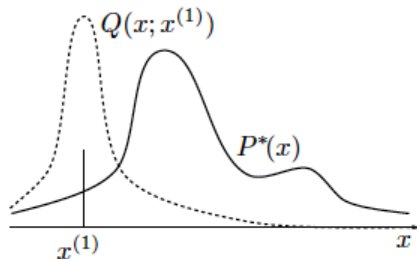
$$p(x) = \frac{p^*(x)}{Z}$$

where Z is a constant

The Metropolis-Hastings Method

Initially

- ▶ Have an initial state $x^{(1)}$
- ▶ Define a proposal density $Q(x'; x^{(t)})$ depending on the current state $x^{(t)}$



- ▶ Must be able to draw samples from $Q(x'; x^{(t)})$

The Metropolis-Hastings Method

At each iteration

- ▶ A tentative new state x' is generated from the proposal density $Q(x'; x^{(t)})$
- ▶ Compute

$$a = \min\left(1, \frac{p^*(x')Q(x^{(t)}; x')}{p^*(x^{(t)})Q(x'; x^{(t)})}\right)$$

- ▶ Accept new state x' with probability a
- ▶ Set

$$x^{(t+1)} = \begin{cases} x' & \text{if state is accepted} \\ x^{(t)} & \text{if state is not accepted} \end{cases}$$

Convergence

- ▶ For any Q s.t. $Q(x'; x) > 0 \forall x, x'$ as $t \rightarrow \infty$

The probability distribution of $x^{(t)}$ tends to $p(x) = p^*(x)/Z$

Gibbs Sampling

Given a state $x^{(t)} \in \mathbb{R}^p$, generate a new state with

$$x_1^{(t+1)} \sim p(x_1 \mid x_2^{(t)}, x_3^{(t)}, \dots, x_p^{(t)})$$

$$x_2^{(t+1)} \sim p(x_2 \mid x_1^{(t+1)}, x_3^{(t)}, \dots, x_p^{(t)})$$

$$x_3^{(t+1)} \sim p(x_3 \mid x_1^{(t+1)}, x_2^{(t+1)}, \dots, x_p^{(t)})$$

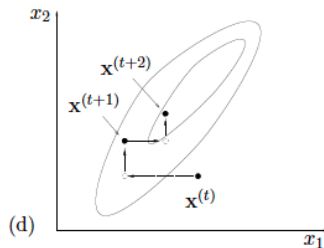
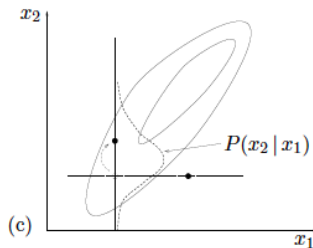
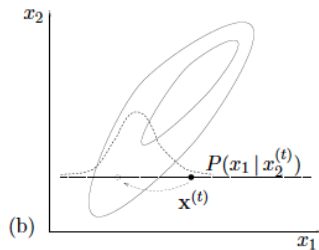
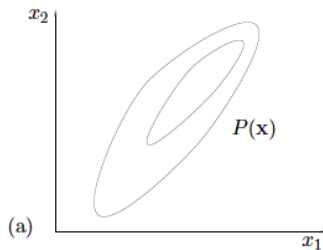
where it is assumed that we can generate samples from $p(x_i \mid \{x_j\}_{j \neq i})$

Convergence

- ▶ Gibbs sampling is a Metropolis method
- ▶ The probability distribution of $x^{(t)}$ tends to $p(x)$ as $t \rightarrow \infty$, as long as $p(x)$ does not have pathological properties

Gibbs Sampling

Two dimensional example



Evolution of Markov Chain

Markov chain defined by an initial $p^{(0)}(x)$ and a transition probability $T(x'; x)$

Let $p^{(t)}(x)$ be the pdf of the state after t applications of the Markov chain

The pdf of the state at the $(t + 1)_{th}$ iteration of the Markov Chain is given by

$$p^{(t+1)}(x') = \int_x T(x'; x) p^{(t)}(x) dx$$

Want to find a chain s.t. as $t \rightarrow \infty$ then $p^{(t)}(x) \rightarrow p(x)$

Evolution of Markov Chain

When designing a MCMC method, construct a chain with the following properties

- ▶ $p(x)$ is an invariant distribution of the chain

$$p(x') = \int_x T(x'; x)p(x)dx$$

- ▶ The chain is ergodic

$$p^{(t)}(x) \rightarrow p(x) \text{ as } t \rightarrow \infty \text{ for any } p^{(0)}(x)$$

Gibbs Sampling for GMM

Close connection between Gibbs sampling and the EM algorithm in exponential family models

Let

- ▶ the parameter θ of the distribution
- ▶ latent / missing data Z^m

be parameter for Gibbs sampler

To estimate the parameter of a GMM at each iteration

- ▶ $\Delta_i^{(t+1)} \sim p(\Delta_i \mid \theta^{(t)}, Z)$ for $i = 1, \dots, n$
- ▶ $\theta^{(t+1)} \sim p(\theta \mid \Delta^{(t+1)}, Z)$

where $\Delta_i \in \{1, \dots, K\}$ represents which component training example i is assigned to

Bagging

Bagging

Starting point

- ▶ Training set $Z = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- ▶ $\hat{f}(x)$ be the prediction at input x learned from Z

Goal

- ▶ Obtain a prediction at input x with lower variance than $\hat{f}(x)$

How - Bootstrap aggregation a.k.a. Bagging

- ▶ Obtain bootstrap samples $Z^{*1}, \dots, Z^{*B}$
- ▶ For each Z^{*b} , fit the model and get prediction $\hat{f}^{*b}(x)$
- ▶ The bagged estimate is then

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

Bagged Estimates

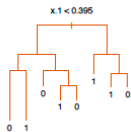
$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

- ▶ Let $\hat{f}(x)$ denote the prediction at input x learned from Z
- ▶ $\hat{f}_{bag}(x)$ differs from $\hat{f}(x)$ when the fitted f is a non-linear or adaptive function of the data

Bagging - Example

- ▶ $N = 30$ training examples with two classes and $p = 5$
- ▶ Each feature is $\mathcal{N}(0, 1)$ with pairwise correlation 0.95
- ▶ The response Y was generated according to $P(Y = 1 \mid x_1 \leq 0.5) = 0.2$ and $P(Y = 1 \mid X_1 > 0.5) = 0.8$
- ▶ Test sample of size 2000 was generated
- ▶ The base classifier, $\hat{f}$, is a classification tree
- ▶ $B = 200$

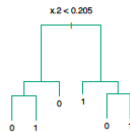
Original Tree



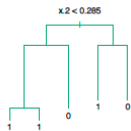
b = 1



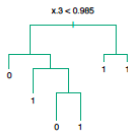
b = 2



b = 3



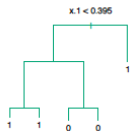
b = 4



b = 5



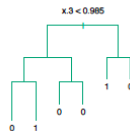
b = 6



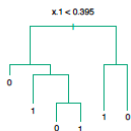
b = 7



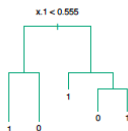
b = 8



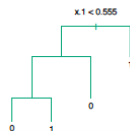
b = 9

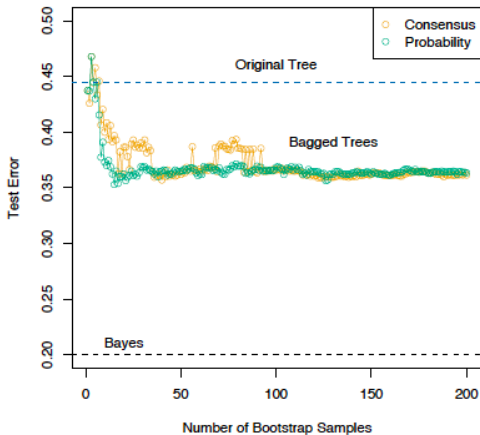


b = 10



b = 11





- ▶ Bag the 0, 1 decision returned by each tree
- ▶ Bag the $P(y = 0 \mid X)$, $P(y = 1 \mid x)$ returned by each tree

Bagging for Classification and 0-1 Loss

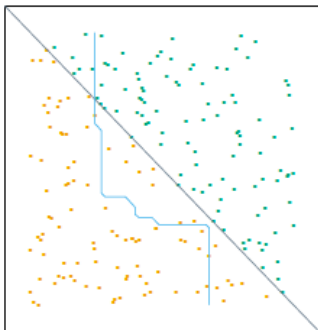
Squared error loss

- ▶ Bagging can dramatically reduce the variance of unstable procedures, leading to improved prediction

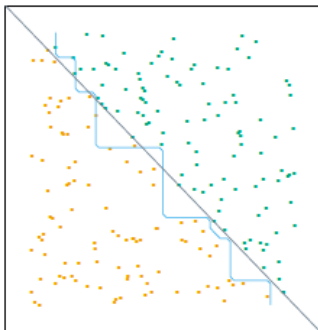
Classification with 0-1 loss

- ▶ Bagging a good classifier can make it better
- ▶ Bagging a bad classifier can make things worse
- ▶ Can understand the bagging effect in terms of a consensus of independent weak learners or the wisdom of crowds

Bagged Decision Rule



Boosted Decision Rule



- ▶ $\hat{f}(x)$ can either be an oriented vertical or horizontal line
- ▶ In this case, bagging the $\hat{f}^{*b}(x)$ gives some gain but not as much as boosting

Model Averaging & Stacking

Bayesian Model Averaging

Starting point

- ▶ Training data $X = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- ▶ A set of candidate models $M_1, \dots, M_M$

Goal

- ▶ Estimate quantity ζ - usually a prediction of $f(x)$ at x

A Bayesian solution

- ▶ The posterior distribution of ζ

$$p(\zeta \mid Z) = \sum_{m=1}^M p(\zeta \mid M_m, Z) P(M_m \mid Z)$$

- ▶ Posterior mean

$$E[\zeta \mid Z] = \sum_{m=1}^M E[\zeta \mid M_m, Z] P(M_m \mid Z)$$

Bayesian Model Averaging

$$E[\zeta \mid Z] = \sum_{m=1}^M E[\zeta \mid M_m, Z] P(M_m \mid Z)$$

- ▶ Committee method

$$P(M_m \mid Z) \approx \frac{1}{M}$$

- ▶ BIC approach

$$P(M_m \mid Z) \approx -2\loglik + d_m \log(n)$$

- ▶ Hardcore Bayesian

$$\begin{aligned} P(M_m \mid Z) &\propto P(M_m) p(Z \mid M_m) \\ &\propto P(M_m) \int p(Z \mid \theta, M_m) p(\theta \mid M_m) d\theta_m \end{aligned}$$

Frequentist Model Averaging

Starting point

- Predictions $\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_M(x)$

Goal

- For squared error loss, find weights $\omega = (\omega_1, \dots, \omega_M)$ s.t.

$$\hat{\omega} = \operatorname{argmin}_{\omega} E_{P_{Y|X=x}} \left(Y - \sum_{m=1}^M \omega_m \hat{f}_m(x) \right)^2$$

Solution: population linear regression of Y on $\hat{F}(x)$

$$\hat{\omega} = E_P[\hat{F}(x)\hat{F}(x)^{\top}]^{-1} E_P[\hat{F}(x)Y]$$

where $\hat{F}(x) \equiv [\hat{f}_1(x), \dots, \hat{f}_M(x)]^{\top}$

Frequentist Model Averaging

$$\hat{\omega} = E_P[\hat{F}(x)\hat{F}(x)^\top]^{-1}E_P[\hat{F}(x)Y]$$

For this $\hat{\omega}$, the full regression model has smaller error than any single model

$$E_P\left(Y - \sum_{m=1}^M \omega_m \hat{f}_m(x)\right)^2 \leq E_P\left(Y - \hat{f}_m(x)\right)^2 \quad \forall m$$

Combining models never makes things worse (at population level)

But cannot estimate the population $\hat{\omega}$. Any thoughts?

Stacked Generalization

Denote $\hat{f}_m^{-i}(x)$ as the prediction at x using

- ▶ the m_{th} model
- ▶ learnt from the dataset with i_{th} training example removed

Then the stacking weights are given by

$$\hat{\omega}^{st} = \underset{\omega}{argmin} \sum_{i=1}^n \left(y_i - \sum_{m=1}^M \omega_m \hat{f}_m^{-i}(x_i) \right)^2$$

The final prediction at point x is

$$\sum_m \hat{\omega}_m^{st} \hat{f}_m(x)$$

Stacked Generalization

Better results by forcing $\hat{\omega}_m^{st}$ to be ≥ 0 and sum to 1

Stacking and model selection via leave-one-out cross-validation are closely related

Can apply stacking to other non-linear methods to combine predictions from different models

Stochastic Search: Bumping

Bumping

General Approach

► Draw bootstrap samples $Z^{*1}, \dots, Z^{*B}$

► For $b = 1, \dots, B$

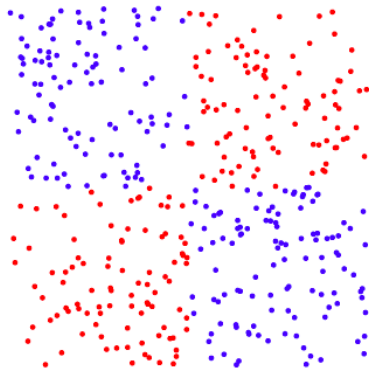
Fit the model to Z^{*b} giving $\hat{f}^{*b}(x)$

► Choose the model obtained from bootstrap sample $\hat{b}$ which minimizes the training error

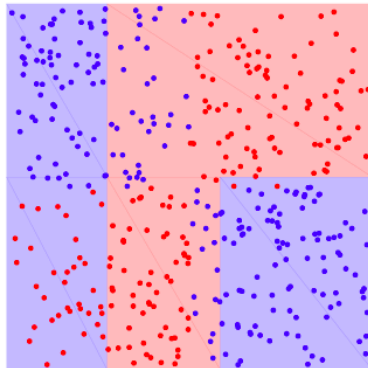
$$\hat{b} = \operatorname{argmin}_b \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}^{*b}(x_i) \right)^2$$

The model predictions are then $\hat{f}^{*\hat{b}}(x)$

Bumping Example



Training data



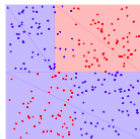
$\hat{f}(x)$ using all training data

Classification using decision tree

- Force tree to have at least 80 points in each leaf

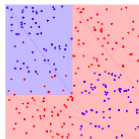
Bumping Example

$$\hat{f}^{*1}(x)$$



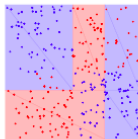
0.2775

$$\hat{f}^{*2}(x)$$



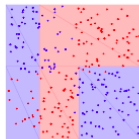
0.3425

$$\hat{f}^{*3}(x)$$



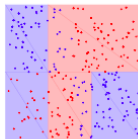
0.2725

$$\hat{f}^{*4}(x)$$



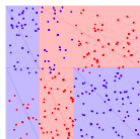
0.2050

$$\hat{f}^{*5}(x)$$



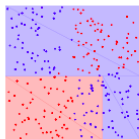
0.2550

$$\hat{f}^{*6}(x)$$



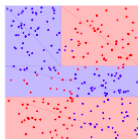
0.1950

$$\hat{f}^{*7}(x)$$



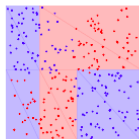
0.3475

$$\hat{f}^{*8}(x)$$



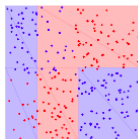
0.2550

$$\hat{f}^{*9}(x)$$



0.1950

$$\hat{f}^{*10}(x)$$

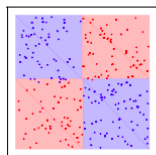


0.2100

Bootstrap sample training data and fit

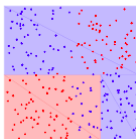
Bumping Example

$$\hat{f}^{*11}(x)$$



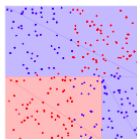
0.0325

$$\hat{f}^{*12}(x)$$



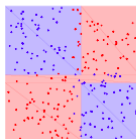
0.3450

$$\hat{f}^{*13}(x)$$



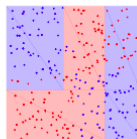
0.3450

$$\hat{f}^{*14}(x)$$



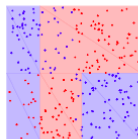
0.1025

$$\hat{f}^{*15}(x)$$



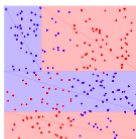
0.2750

$$\hat{f}^{*16}(x)$$



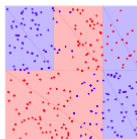
0.2000

$$\hat{f}^{*17}(x)$$



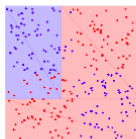
0.3100

$$\hat{f}^{*18}(x)$$



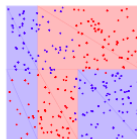
0.2775

$$\hat{f}^{*19}(x)$$



0.3425

$$\hat{f}^{*20}(x)$$



0.2000

Bootstrap sample training data and fit

Bumping

- ▶ Bumping perturbs the training data
- ▶ Therefore explore different areas of the model space
- ▶ Must ensure the complexity of each model fit is comparable